

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ ІВАНА ФРАНКА

ЗВІТ

з дисципліни “Функціональне програмування”

на тему « Основи функціонального програмування у Python »

Виконав:

Студент 2 курсу

групи ФсП-23

МАГОРА Максим

Перевірив:

СВАЛЕБА Сергій Андрійович

- **Номер та тема варіанта:** Варіант №11 («Мобільний тариф»).

DOCX+ 1

- **Мета роботи:**
 - Зрозуміти, що таке чиста функція і референтна прозорість;
 - Навчитися виявляти та ізолювати побічні ефекти;
 - Переписати імперативний фрагмент у функціональному стилі;
 - Застосувати `typing.Callable` для параметризації обчислень.

ВИКОНАННЯ РОБОТИ

```
# Лабораторна робота №1. Варіант 11: Мобільний тариф
```

Опис проекту

Проект реалізує функціональний підхід до розрахунку місячної вартості мобільного тарифу. Обчислювальне ядро не містить мутацій даних, глобального стану та I/O операцій.

Структура

- ``core.py``: Чисте функціональне ядро.
- ``app.py``: Оболонка для виводу результатів (I/O).
- ``tests/test_core.py``: Pytest-тести на детермінованість, відсутність мутацій та коректність політик.

Запуск

1. Встановлення залежностей:

```
```bash
pip install -r requirements.txt
```

```
from typing import Callable, List, Dict, Any, TypedDict
```

```
class Usage(TypedDict):
```

```
 minutes: float
```

```
 sms: int
```

```
 data_gb: float
```

```
class MobileUser(TypedDict):
```

```
 id: int
```

```
 name: str
```

```
 plan: str
```

```
 usage: Usage
```

```
 bonus_points: int
```

```

TariffRuleFn = Callable[[Usage], float]
BonusRuleFn = Callable[[float, int], float]

def calculate_usage_cost(usage: Usage, minute_rate: float, sms_rate: float,
data_rate_per_gb: float) -> float:
 """Обчислює базову вартість спожитих послуг без мутації даних."""
 return (
 usage["minutes"] * minute_rate
 + usage["sms"] * sms_rate
 + usage["data_gb"] * data_rate_per_gb
)

def apply_loyalty_discount(cost: float, bonus_points: int) -> float:
 """Застосовує знижку за бонуси (1 бонус = 0.5 грн знижки, але не більше 30% суми)."""
 max_discount = cost * 0.3
 discount = min(bonus_points * 0.5, max_discount)
 return max(0.0, cost - discount)

def calculate_user_monthly_cost(
 user: MobileUser,
 tariff_fn: TariffRuleFn,
 bonus_fn: BonusRuleFn
) -> MobileUser:
 """Створює НОВИЙ словник користувача із розрахунком поля monthly_cost."""
 base_cost = tariff_fn(user["usage"])
 final_cost = bonus_fn(base_cost, user["bonus_points"])

 # Створення нового об'єкта без мутації оригінального
 return {
 **user,
 "monthly_cost": round(final_cost, 2)
 }

def make_mobile_processor(
 tariff_fn: TariffRuleFn,
 bonus_fn: BonusRuleFn
) -> Callable[[List[MobileUser]], Dict[str, Any]]:
 """Фабрика функцій вищого порядку для обробки списку користувачів."""
 def process(users: List[MobileUser]) -> Dict[str, Any]:
 processed_users = [
 calculate_user_monthly_cost(u, tariff_fn, bonus_fn)
 for u in users
]
 total_revenue = sum(u["monthly_cost"] for u in processed_users)
 return {
 "count": len(processed_users),
 "total_revenue": round(total_revenue, 2),
 "users": processed_users
 }

```

```

 }
 return process
from core import (
 MobileUser,
 calculate_usage_cost,
 apply_loyalty_discount,
 make_mobile_processor
)

def render_report(result: dict) -> None:
 """Друк квитанцій та підсумків (ізолюваний I/O)."""
 print("=" * 45)
 print(" РОЗРАХУНОК МОБІЛЬНИХ ТАРИФІВ")
 print("=" * 45)
 for u in result["users"]:
 print(f"Користувач: {u['name']} (ID: {u['id']})")
 print(f" Тариф: {u['plan']}")
 print(f" Використано: {u['usage']['minutes']} хв, {u['usage']['sms']} SMS, {u['usage']['data_gb']} ГБ")
 print(f" Місячна вартість (monthly_cost): {u['monthly_cost']} грн")
 print("-" * 45)
 print(f"Усього оброблено: {result['count']} абонентів")
 print(f"Загальна виручка: {result['total_revenue']} грн")
 print("=" * 45)

def main():
 sample_users: list[MobileUser] = [
 {
 "id": 1,
 "name": "Олексій",
 "plan": "Smart",
 "usage": {"minutes": 150, "sms": 20, "data_gb": 12.5},
 "bonus_points": 50,
 },
 {
 "id": 2,
 "name": "Марія",
 "plan": "Unlim",
 "usage": {"minutes": 400, "sms": 5, "data_gb": 35.0},
 "bonus_points": 120,
 },
]

 # Визначення політик через lambda / Callable
 smart_tariff = lambda u: calculate_usage_cost(u, minute_rate=0.8, sms_rate=0.3, data_rate_per_gb=10.0)
 processor = make_mobile_processor(smart_tariff, apply_loyalty_discount)

```

```
 result = processor(sample_users)
 render_report(result)

if __name__ == "__main__":
 main()
```

## pytest

## mypy

## black

## ruff

```
import pytest
from copy import deepcopy
from core import (
 MobileUser,
 calculate_usage_cost,
 apply_loyalty_discount,
 calculate_user_monthly_cost,
 make_mobile_processor
)

@pytest.fixture
def sample_user() -> MobileUser:
 return {
 "id": 101,
 "name": "Тестовий Користувач",
 "plan": "Basic",
 "usage": {"minutes": 100, "sms": 10, "data_gb": 5.0},
 "bonus_points": 20
 }

def test_referential_transparency(sample_user):
 """Перевірка детермінованості: однаковий вхід -> однаковий вихід."""
 tariff = lambda u: calculate_usage_cost(u, 1.0, 0.5, 5.0)
 res1 = calculate_user_monthly_cost(sample_user, tariff, apply_loyalty_discount)
 res2 = calculate_user_monthly_cost(sample_user, tariff, apply_loyalty_discount)
 assert res1 == res2

def test_no_mutation(sample_user):
 """Перевірка відсутності мутацій вхідного об'єкта."""
 original = deepcopy(sample_user)
 tariff = lambda u: calculate_usage_cost(u, 1.0, 0.5, 5.0)
 calculate_user_monthly_cost(sample_user, tariff, apply_loyalty_discount)

 assert sample_user == original
```

```

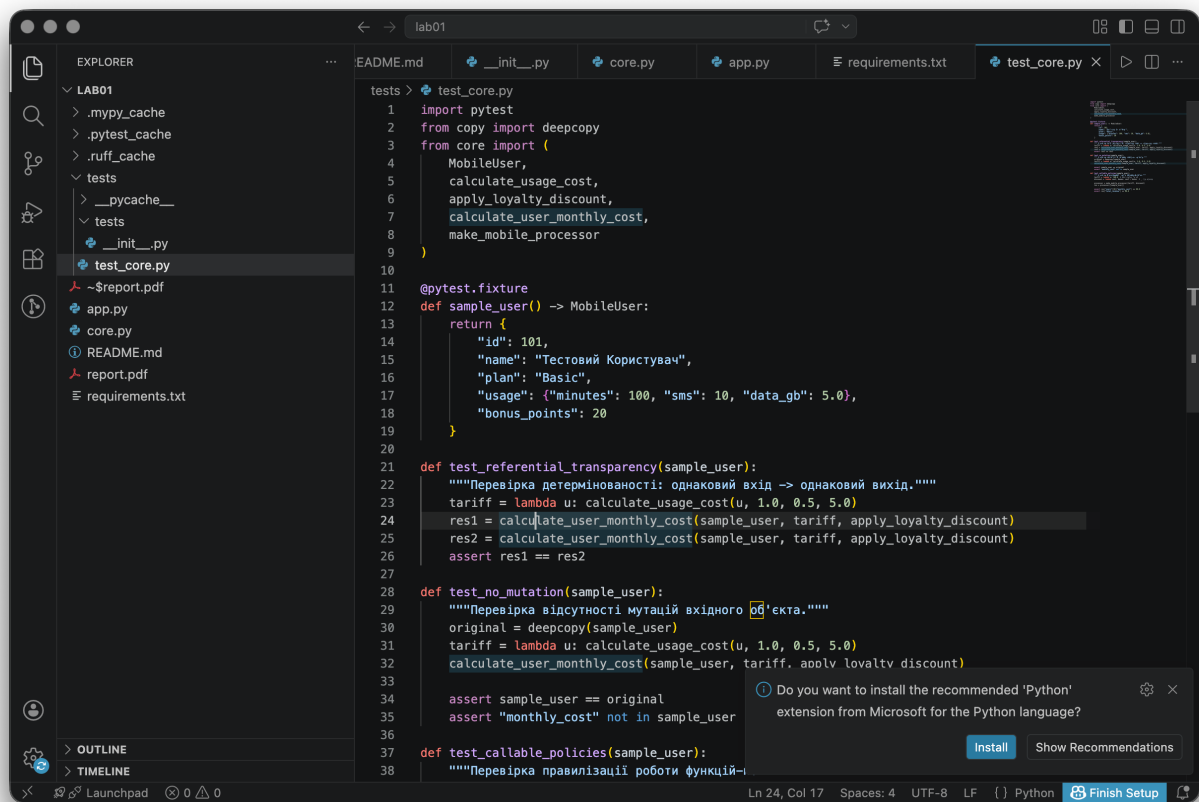
 assert "monthly_cost" not in sample_user

def test_callable_policies(sample_user):
 """Перевірка правилізації роботи функцій-політик."""
 tariff = lambda u: 100.0 # Фіксований тариф
 discount = lambda cost, bonus: cost - bonus # Пряма знижка

 processor = make_mobile_processor(tariff, discount)
 res = processor([sample_user])

 assert res["users"][0]["monthly_cost"] == 80.0
 assert res["total_revenue"] == 80.0

```



## ВИСНОВКИ

У результаті виконання лабораторної роботи:

- зрозумів, що таке чиста функція і референтна прозорість;
- навчився виявляти та ізолювати побічні ефекти;
- переписав імперативний фрагмент у функціональному стилі;
- реалізував чисте обчислювальне ядро з параметризацією через `typing.Callable` та покрити його unit-тестами.